

Music Knowledge Graph (MusicKG)

Intelligent Semantic System with Automated Inference

Course: Semantic Web 2025/26

Technologies: Python • Django • RDF • GraphDB • SPARQL • React

Dataset: 30000 Spotify Songs from Kaggle

[112793] Cláudia Seabra

June 30th, 2026

1. Introduction	3
2. Definition of the Ontology (RDFS and OWL)	3
2.1. Class Hierarchy and Automated Sub-classing.....	3
2.2. Object Properties and OWL Constraints	3
2.3. Datatype Properties and Restrictions	4
2.4. Protégé Verification.....	5
3. SPIN Inference Set.....	5
3.1. Rules Logic.....	6
3.2. Automated Graph Maintenance.....	6
4. New Operations on the Data (SPARQL)	6
4.1. Intelligent Data Normalization and URI Integrity.....	6
4.2. Automated Graph Maintenance and Referential Integrity.....	6
4.3. Materialized Reasoning & Logic Decoupling.....	7
4.4. Operational Interface Control	7
5. Use and Integration of Data from Wikidata and DBpedia	7
5.1. Dynamic Enrichment Architecture.....	7
6. Publication of Semantic Data through RDFa and Microformats	8
6.1. Vocabulary Selection and Ontological Mapping	8
6.2. Nested Semantic Integration	8
6.3. Frontend Implementation Strategy.....	9
7. Conclusions	9
8. Configuration and Execution.....	10
8.1. System Requirements	10
8.2. Installation & Setup	10
8.3. Verification	10

1. Introduction

The Music Knowledge Graph (MusicKG) application was initially developed to transform a relational Spotify dataset (32 833 tracks) into a queryable RDF-based semantic repository. While the first phase successfully established the foundational architecture, it operated as a static database.

The objective of this project is to evolve this repository into an Intelligent Semantic System. This phase focuses on deep ontological modeling, automated reasoning, and global interoperability. By integrating OWL restrictions, custom SPIN-based inference rules, dynamic federation with DBpedia, and frontend RDFa annotations, the system now acts as a network capable of automatically categorizing entities, expanding its own dataset, and publishing its data in machine-readable formats.

2. Definition of the Ontology (RDFS and OWL)

The ontology was refined to accommodate inferred classes and enriched metadata.

2.1. Class Hierarchy and Automated Sub-classing

The ontology establishes a foundational set of Base Classes to represent the explicit entities within the dataset, alongside a layer of Defined/Inferred Subclasses. These subclasses allow reasoners to automatically categorize instances based on specific property conditions without manual data entry.

Base Classes:

- `music:Artist`: The central entity representing a music creator or band.
- `music:Album`: A collection of tracks released as a single project.
- `music:Track`: An individual musical recording.
- `music:Genre`: A stylistic category (e.g., pop, rock, edm) linked to artists and tracks.
- `music:AudioFeatures`: A specialized node containing the technical acoustic metrics of a track, separated from the track itself to allow for dimensional modeling.

Inferred Classes

The system uses specific classes for reasoning, such as `music:HighEnergyTrack`, `music:PopularTrack`, and temporal era classes: `music:ClassicAlbum`, `music:ModernAlbum`, and `music:TransitionAlbum`. The latter acts as a semantic buffer for the 2000–2009 period.

2.2. Object Properties and OWL Constraints

The ontology employs a strictly defined set of Object and Datatype properties to enforce structural integrity and facilitate automatic reasoning. OWL property axioms were used to reduce data redundancy and ensure logical consistency across the graph.

- `music:hasAlbum`: Establishes a formal relationship between `music:Artist` (Domain) and `music:Album` (Range).

- `music:hasTrack` & `music:inAlbum` (Inverse Properties): These properties are defined as `owl:inverseOf` each other. By asserting that a track is `music:inAlbum`, the reasoner automatically infers that the album `music:hasTrack` (that specific recording). This way there's no need for manual bidirectional data entry.
- `music:performedBy` & `music:performs` (Inverse Properties): These properties link tracks to their artists. The declaration of `music:performs` as the inverse of `music:performedBy` ensures that querying an artist's discography is semantically consistent regardless of the traversal direction.
- `music:inGenre`: A property linking both `music:Track` and `music:Artist` to `music:Genre` nodes, facilitating cross-entity genre-based navigation.
- `music:similarTo` (Symmetric Property): To optimize similarity storage, this relation is defined as an `owl:SymmetricProperty`. If the graph asserts that Artist A is similar to Artist B, the reasoner automatically infers that Artist B is similar to Artist A, effectively halving the number of stored triples required for similarity edges.
- `music:hasAudioFeatures`: A strictly controlled property that links a `music:Track` to a unique `music:AudioProfile` node, centralizing acoustic metrics like energy, danceability, and valence.

2.3. Datatype Properties and Restrictions

To ensure structural integrity and minimize data anomalies, it was defined specific Datatype Properties and applied OWL functional constraints. This rigorous typing guarantees that the Knowledge Graph remains consistent even after merging disparate fact files.

- **Semantic Data Mapping:** Literal values are strictly typed using XML Schema Definition (XSD) primitives:
 - Descriptive Strings (`xsd:string`): Captures textual metadata including `music:artistName`, `music:albumName`, `music:trackName`, and the federated `music:dbpediaAbstract`.
 - Acoustic Metrics (`xsd:float`): Encapsulates normalized Spotify features such as `music:energy`, `music:danceability`, `music:valence`, `music:tempo`, and `music:loudness`.
 - Temporal & Scoring (`xsd:integer`): Manages range-based data, specifically `music:popularity` (0–100 scale) and `music:releaseYear`.
- **Functional Property Constraints:** `owl:FunctionalProperty` is utilized to enforce cardinality constraints on key attributes.
 - For example, `music:releaseYear` is declared as a Functional Property, explicitly dictating that a `music:Album` resource can possess only one release year value.

This way, it is ensured that when the independent *ontology.ttl* and *facts_only.nt* files are merged, the reasoner maintains absolute logical coherence.

2.4. Protégé Verification

To ensure the logical correctness of the ontology, the *ontology.ttl* file was formally verified using the Protégé ontology editor.

The ontology was loaded to verify the class hierarchy, object property domain/range restrictions, and the successful application of *owl:inverseOf* and *owl:FunctionalProperty* axioms.

The Hermit 1.4.3.456 reasoner was used to validate the logical consistency of the model. The reasoner was successfully started, reporting no unsatisfiable classes, which confirms that the domain constraints are logically sound and free of contradictions. The integrated RDF file was also inspected to visualize the property flow between Artist, Album, and Track instances.

Figure 1 below illustrates the Protégé interface during the validation of the MusicKG ontology, highlighting the active reasoner and the successfully parsed metrics (58 axioms and 29 logical axioms).

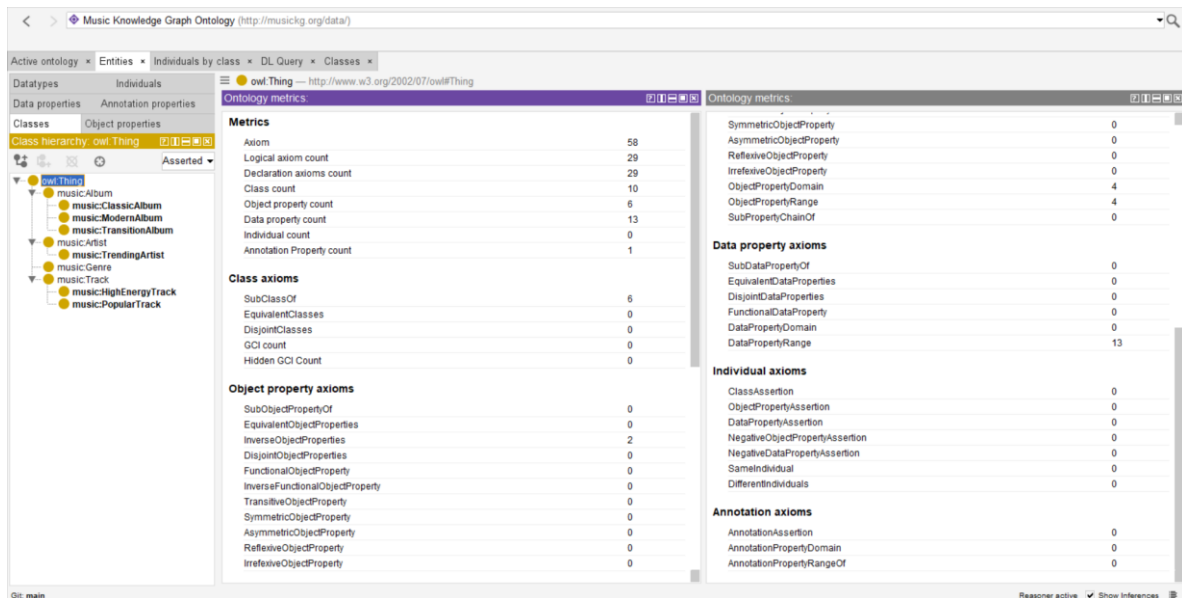


Figure 1: Ontology validation in Protégé using the Hermit reasoner.

3. SPIN Inference Set

While standard Description Logic (DL) reasoners handle basic structural hierarchies (e.g., transitive and inverse properties), these are inherently limited when dealing with complex mathematics and cross-entity aggregations. To resolve this, a set of custom inference rules was developed using SPARQL Inferencing Notation (SPIN) principles.

In this architecture, the SPIN rules are isolated within an independent Python module (*spin_rules.py*).

3.1. Rules Logic

- High Energy: Tracks with energy > 0.8 are typed `music:HighEnergyTrack`.
- Popularity: Tracks with popularity >= 70 are typed `music:PopularTrack`.
- Temporal Era: Albums are dynamically classified based on `music:releaseYear`.
- Trending Artist: Aggregates popularity via SPARQL AVG to identify trending artists.

3.2. Automated Graph Maintenance

A pre-inference DELETE cleanup query was implemented in *rdf_store.py* to remove conflicting era classifications before re-materializing inferred triples.

4. New Operations on the Data (SPARQL)

The architecture of the MusicKG system evolved in this phase from a static RDF dataset into a dynamic, read-write Knowledge Graph. By leveraging the GraphDB HTTP API, the backend (*rdf_store.py*) facilitates real-time graph evolution. This transition was supported by an upgraded *sparql_queries.py* module, which now handles the lifecycle management of graph resources as well.

4.1. Intelligent Data Normalization and URI Integrity

A primary challenge identified was URI collision, particularly with common titles such as "Greatest Hits". To ensure unique resource identification, a more robust URI normalization strategy is implemented:

- Slug Generation: Resource URIs were restructured using a `{ArtistName}_{AlbumName}` pattern. This ensures that every album entity is globally unique within the graph, regardless of its title.
- Safe Encoding: I utilized Python's `urllib.parse.quote` to clean strings, creating URL-safe slugs (e.g., converting spaces and special characters into URI-compliant formats). This mitigated and prevented HTTP 404 errors during client-side navigation.

4.2. Automated Graph Maintenance and Referential Integrity

To maintain the graph's health without manual intervention, the system employs a automated cleanup through SPARQL.

- Using FILTER NOT EXISTS clauses, the maintenance logic verifies that an album node is only preserved if it maintains an active `music:hasAlbum` link from an Artist or holds at least one child Track node. This prevents remaining semantic entities that could otherwise lead to null pointer exceptions or broken semantic chains in the frontend.

4.3. Materialized Reasoning & Logic Decoupling

SPIN/SPARQL Synchronization

The system maintains a separation between "raw facts" (imported from CSV) and "inferred knowledge" (generated by SPIN rules).

Before any reasoning pass, the system executes an automated DELETE operation to remove previously inferred class assertions (e.g., ClassicAlbum, PopularTrack). This ensures the reasoner operates on a clean state and prevents logical contradictions (such as an album being classified simultaneously as both "Classic" and "Modern" due to stale data).

4.4. Operational Interface Control

Read Operations

The application now supports complex analytical queries, including pagination, multi-level aggregation, and audio feature distribution analysis.

Write Operations

The API exposes secure POST endpoints for CRUD operations. These endpoints use INSERT DATA and DELETE/INSERT SPARQL commands to ensure that every UI interaction (like editing a track's energy level) is reflected immediately as a triple modification in GraphDB.

5. Use and Integration of Data from Wikidata and DBpedia

The system employs a Dynamic Federated Enrichment Engine (located in *music_graph/linked_data.py*) designed to bridge the gap between the local dataset and the broader Semantic Web. By integrating live data from DBpedia and Wikidata, the application transforms isolated local entities into interconnected nodes within the global Linked Data cloud.

5.1. Dynamic Enrichment Architecture

Instead of a static batch-processing approach, the system utilizes an on-demand enrichment strategy triggered by user navigation. The architecture functions as follows:

- When a user accesses an ArtistDetail view, the backend performs a proactive introspection of the local RDF node.
- The system queries the local GraphDB instance to check for the presence of semantic properties, specifically `music:dbpediaAbstract` and `music:imageUrl`.
- If these properties are missing, the system initiates a two-stage federated fetch:
 1. DBpedia Lookup: Utilizes the DBpedia Lookup API to resolve the local `artistName` to a canonical DBpedia resource URI. It then executes a SPARQL query against the DBpedia endpoint to extract the English abstract and the corresponding `owl:sameAs` Wikidata identifier.

2. Wikidata Resolution: If a Wikidata URI is retrieved, the engine executes a secondary SPARQL query against the Wikidata Query Service (WDQS) to resolve complex metadata, such as high-resolution image URLs and the artist's country of origin.
- Once fetched and sanitized, the engine generates new RDF triples. These are persisted locally in `data/enrichment.ttl` and ingested into the live repository via the GraphDB statements endpoint.

6. Publication of Semantic Data through RDFa and Microformats

To maximize the interoperability of the application, the frontend implements a semantic publishing layer. By embedding machine-readable metadata directly into the HTML rendering via RDFa (Resource Description Framework in Attributes), it is ensured that the MusicKG content is visually accessible to users.

6.1. Vocabulary Selection and Ontological Mapping

Our internal `music:` namespace was aligned with Schema.org, the industry-standard vocabulary for structured web data.

- `music:Artist` maps to `schema:MusicGroup`: Annotating artist profiles with this type allows aggregators to index professional music entities and their associated metadata.
- `music:Album` maps to `schema:MusicAlbum`: This enables search engines to recognize the entity as a formal collection of creative works.
- `music:Track` maps to `schema:MusicRecording`: This creates a fine-grained linkage for individual audio performances, surfacing them in rich snippets and voice search results.
- External Identity: We utilized `property="sameAs"` to link local nodes to global identifiers (DBpedia and Wikidata URIs).

6.2. Nested Semantic Integration

The application implements Semantic Nesting, which mirrors the hierarchical relationships defined in the backend Knowledge Graph.

Hierarchical Composition:

- Artist-to-Album: Artist profile pages are annotated with `typeof="MusicGroup"`, and albums are nested as children.
- Album-to-Track: Album detail pages are rendered with `typeof="MusicAlbum"`, containing a list of tracks annotated with `property="hasTrack"` and `typeof="MusicRecording"`.

Intelligent UI Aggregation:

To ensure high-quality semantic data even in search results, the backend `full_text_search` utility employs SAMPLE aggregation in SPARQL. This allows the search results page to dynamically expose the associated `artistName` for both `Track` and `Album` entities.

The frontend renders these results with contextual annotations, ensuring that even in a summary list, a "Track" entity carries its provenance (the performer) semantically.

6.3. Frontend Implementation Strategy

I utilized global attributes (`vocab`, `typeof`, `property`, `resource`) to decorate the DOM to ensure that as the frontend updates (e.g., when a user adds a track or updates an album year), the semantic metadata is automatically synchronized with the underlying RDF graph.

By using the `resource` attribute bound to the unique URI of the node (e.g., `resource="{{album.uri }}"`), ambiguity is prevented and it is ensured that every piece of structured data is tied to the exact node in GraphDB.

7. Conclusions

The second phase of the Music Knowledge Graph (MusicKG) project successfully elevated a static RDF database into a highly dynamic and intelligent semantic ecosystem. By transitioning from a read-only data model to a fully interactive system, the project demonstrates the practical viability of the Semantic Web in modern application development.

The integration of an automated inference layer, driven by SPIN-compatible rules, allowed the system to transcend the limitations of basic RDFS/OWL structures. Furthermore, MusicKG dynamically links local artist nodes to global counterparts in DBpedia and Wikidata via `owl:sameAs` and `music:dbpediaAbstract`.

The integration of RDFa and Schema.org annotations directly within the frontend represents a significant milestone in semantic visibility. By nesting semantic data in the HTML structure, it has been ensured that the graph's internal hierarchies are not just stored in GraphDB but are also machine-interpretable for external search engines.

In summary, this phase of the MusicKG project successfully demonstrates that semantic technologies can support complex user-facing features while maintaining strict logical constraints. Despite the inherent challenges of federated data and logical reasoning, the system achieved a stable, scalable, and validated state.

8. Configuration and Execution

8.1. System Requirements

- Python 3.9+
- GraphDB Free/Standard installed and running locally on port 7200.

8.2. Installation & Setup

Step 1: Install Dependencies and Generate the RDF Graph

```
pip install -r requirements.txt
```

```
python convert_to_rdf.py --csv spotify_songs.csv --data-dir data/
```

This performs the ETL process and generates `facts_only.nt` (for raw data), `ontology.ttl` (for schema definition), and `spin_rules.ttl` (for inference logic).

Step 2: Automated Reasoning and Initialization

```
python music_graph/spin_rules.py --output data/spin_rules.ttl
```

```
python manage.py migrate python manage.py runserver
```

8.3. Verification

Access the application by navigating to `http://127.0.0.1:8000/` in any browser. On the global Statistics Dashboard (Home page), the system will display the real-time connection status (indicating whether it is operating in "GraphDB" mode or the safe "rdflib fallback" mode).